

“Tower of Babel”
**Seleksi Asisten Laboratorium Sistem Paralel dan
Terdistribusi 2026**



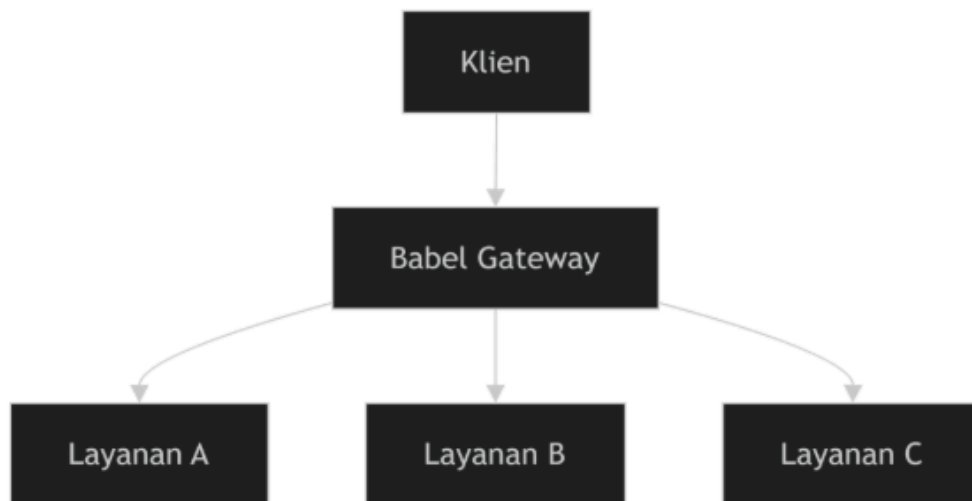
Azri Arzaq Pohan - 13524139

DAFTAR ISI

DAFTAR ISI.....	2
Arsitektur Gateway.....	3
Service Registry.....	6
Strategi Translasi Protokol.....	11
- Translasi Service A.....	12
- Translasi Service B.....	13
- Translasi Service C.....	14
Strategi Routing.....	15
Model Concurrency.....	18
Penanganan Kegagalan.....	20
Keputusan dan Kompromi Teknis.....	21
Keterbatasan Implementasi.....	23

Arsitektur Gateway

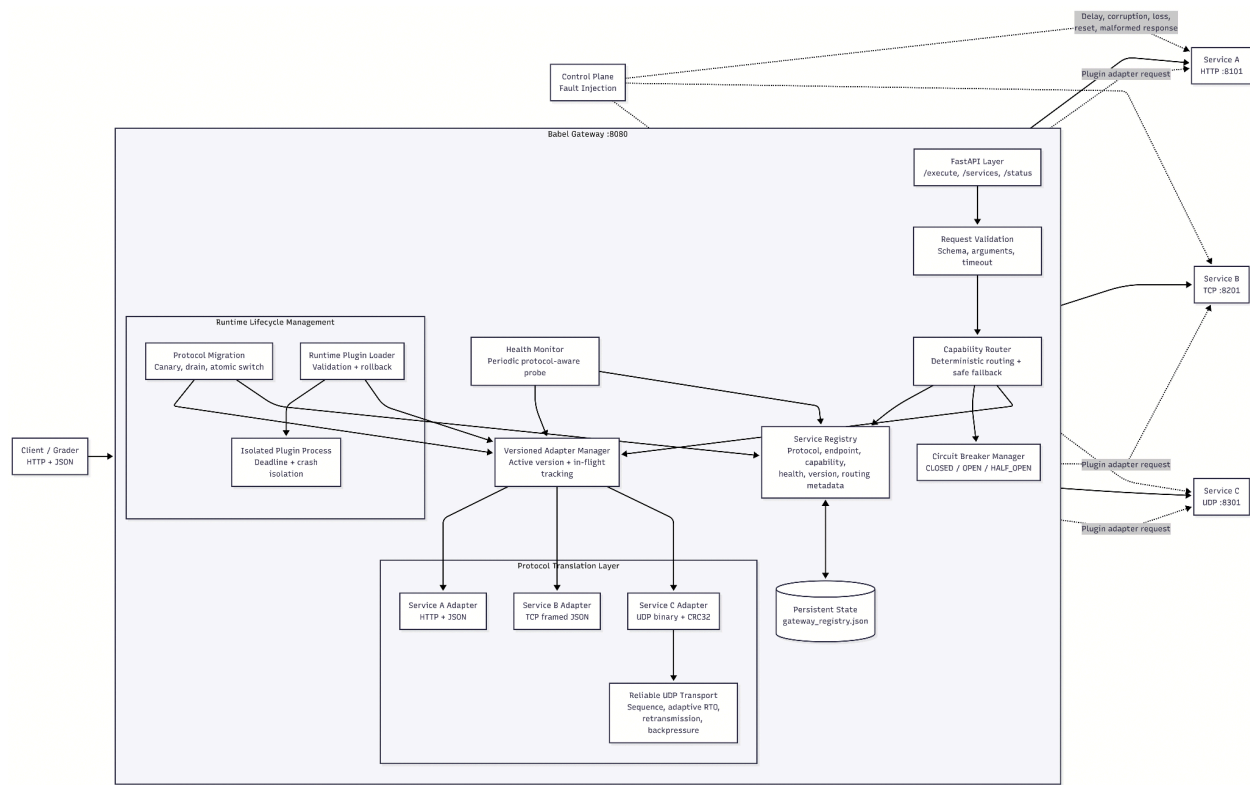
Pada tugas ini, **Tower of Babel** merupakan sebuah sistem terdistribusi yang memiliki beberapa *backend services* dengan masing-masing *service*-nya memiliki protokol komunikasi yang berbeda satu sama lainnya. **Service A** disini menggunakan protokol HTTP/JSON, **Service B** menggunakan raw TCP dengan *binary framing*, sedangkan **Service C** menggunakan UDP dengan protokol khususnya sendiri. Perbedaan-perbedaan tersebut juga membuat *client* tidak bisa untuk berkomunikasi dengan semua *service* menggunakan satu mekanisme yang seragam. Berikut adalah gambaran umumnya.



Gambar 1. Gambaran umum *gateway*

Oleh karena itu, solusi permasalahan yang saya tawarkan disini adalah dengan menggunakan **Babel Gateway** sebagai lapisan *translator* sekaligus penghubungnya. Sehingga *client* cukup untuk berkomunikasi dengan *gateway* melalui satu *interface* HTTP/JSON. Kemudian *gateway* akan memilih *service* yang sesuai, lalu *men-translate* dan mengirim *request* ke protokol *service* tujuan, memvalidasi *response* yang dikirimkan, lalu mengubah hasilnya kembali ke format yang sudah terstandarisasi untuk *client*.

Berikut adalah gambaran arsitektur dari *gateway* yang sudah saya implementasikan.



Gambar 2. Diagram Arsitektur Gateway

Setiap *request* yang masuk terlebih dahulu akan diproses oleh komponen API dan *request validation*. Nah di tahap ini, *gateway* akan memeriksa kelengkapan dan juga validitas dari data-nya, seperti **requestId**, nama operasi, argumen, versi dari protokol, dan batas waktu eksekusinya. Validasi disini akan dilakukan sebelum *request*-nya diteruskan supaya data yang “tidak sesuai” tidak akan bisa mencapai *backend service*. **requestId** disini juga memungkinkan setiap *request* dan *response* untuk ditelusuri serta dilakukan pencocokan selama melewati berbagai protokol.

Setelah berhasil untuk melewati proses validasi, *request* tersebut akan diteruskan menuju komponen *capability-based router*. Tugas dari komponen ini adalah untuk menentukan *backend service* yang paling sesuai berdasarkan operasi yang ingin dijalankan oleh *client*. Saat dilakukan pemilihan, *router* akan membaca informasi dari *service registry*, seperti kemampuan apa saja yang dimiliki oleh setiap *service*-nya, protokol yang digunakan, *health status* dari *service*, prioritas, serta versi adapter yang tersedia. Sehingga *request* nantinya hanya akan diarahkan menuju *service* yang memang mampu menangani operasi tersebut.

Setelah *service* tujuan berhasil ditentukan, *gateway* nantinya akan memilih *adapter* yang sesuai dengan protokol dari *service* tersebut. *Adapter* ini berfungsi sebagai yang *men-translate* antara format *internal gateway* dengan format pesan yang dipahami oleh *backend service*. Untuk **Service A**, *gateway* akan menggunakan *adapter* HTTP/JSON. **Service B** akan diakses menggunakan *adapter* raw TCP dengan *binary framing*, sedangkan **Service C** akan diakses menggunakan **adapter** UDP dengan struktur paket khusus. Sehingga *client* tidak perlu mengetahui detail protokol yang digunakan oleh masing-masing *service*.

Selanjutnya, *adapter* akan mengirimkan *request* yang sudah di-*translate* menuju *backend service* tujuan. Setelah *backend* selesai untuk memproses *request*, *response* yang diterima akan diperiksa kembali oleh *adapter* untuk memastikan bahwa format, versi protokol, identitas *request*, dan isi *response*-nya sesuai. *Response* tersebut kemudian di-*translate* kembali ke dalam format standar *gateway* sebelum dikirimkan kepada *client* melalui HTTP/JSON. Sehingga *client* akan selalu menerima bentuk *response* yang konsisten meskipun *request* sebenarnya itu telah diproses oleh *service* dengan protokol yang berbeda-beda.

Service Registry

Service registry merupakan komponen yang berfungsi sebagai pusat penyimpanan informasi mengenai seluruh *backend service* yang terhubung dengan **Babel Gateway**. Setiap service akan direpresentasikan dalam bentuk **ServiceEntry** yang berisi **service_id**, jenis protokol, **endpoint**, daftar kemampuan, *health status*, versi aktif, dan metadata tambahan. Informasi tersebut nantinya akan digunakan oleh komponen lain agar gateway tidak perlu menyimpan konfigurasi service secara terpisah di setiap bagian sistem.

```
class ServiceEntry:
    def __init__(
        self,
        service_id: str,
        protocol: str,
        endpoint: str,
        capabilities: List[str],
        status: str = "available",
        version: str = "1",
        meta: Optional[Dict[str, Any]] = None,
    ) -> None:
        self.service_id = service_id
        self.protocol = protocol
        self.endpoint = endpoint
        self.capabilities = capabilities
        self.status = status
        self.version = version
        self.meta = meta or {}

    def to_dict(self) -> Dict[str, Any]:
        return {
            "service_id": self.service_id,
            "protocol": self.protocol,
            "endpoint": self.endpoint,
            "capabilities": list(self.capabilities),
            "status": self.status,
            "version": self.version,
            "meta": copy.deepcopy(self.meta),
        }
```

```

def copy(self) -> "ServiceEntry":
    return ServiceEntry.from_dict(self.to_dict())

@classmethod
def from_dict(cls, data: Dict[str, Any]) -> ServiceEntry:
    required = {"service_id", "protocol", "endpoint"}
    missing = required - set(data)
    if missing:
        raise ValueError(f"Registry entry missing fields:
{sorted(missing)}")
    capabilities = data.get("capabilities", [])
    if not isinstance(capabilities, list) or not all(
        isinstance(item, str) and item for item in
capabilities
    ):
        raise ValueError("Registry capabilities must be a
list of non-empty strings")
    status = data.get("status", "unknown")
    if status not in {"available", "unavailable",
"unknown"}:
        raise ValueError(f"Invalid registry health status
'{status}'")
    version = str(data.get("version", "1"))
    if not version.isdigit() or not 1 <= int(version) <=
255:
        raise ValueError(f"Invalid registry protocol version
'{version}'")
    meta = data.get("meta", {})
    if not isinstance(meta, dict):
        raise ValueError("Registry meta must be an object")
    return cls(
        service_id=data["service_id"],
        protocol=data["protocol"],
        endpoint=data["endpoint"],
        capabilities=capabilities,
        status=status,
        version=version,
        meta=meta,

```

)

Ketika *gateway* pertama kali dijalankan, *registry* disini akan menyediakan konfigurasi bawaan untuk **Service A**, **Service B**, dan **Service C**. Setelah itu, *gateway* akan mencoba membaca konfigurasi *registry* yang sebelumnya disimpan pada file **gateway_registry.json**. Jika file tersebut belum tersedia, maka konfigurasi bawaan nantinya akan digunakan dan langsung disimpan sebagai state awal. Jika tersedia, maka nanti isinya akan divalidasi terlebih dahulu sebelum digunakan. Mekanisme ini memungkinkan konfigurasi *service*, versi *adapter*, dan *metadata routing* tetap tersedia setelah *gateway* mengalami *restart*.

Berikut adalah contoh isi dari **gateway_registry.json** ketika *gateway*-nya dijalankan melalui **docker compose**.

```
"backends": {
  "service-a": "available",
  "service-b": "available",
  "service-c": "available"
},
"circuit_breakers": {},
"gateway_id": "candidate-gateway",
"services": [
  {
    "capabilities": [
      "echo",
      "metadata",
      "uppercase"
    ],
    "endpoint": "http://service-a:8101",
    "protocol": "http-json",
    "service_id": "service-a",
    "status": "available",
    "supported_versions": [
      "1"
    ],
    "version": "1"
  },
  {
```



```
{
  "capabilities": [
    "echo",
    "metadata",
    "reverse",
    "sum",
    "uppercase"
  ],
  "endpoint": "service-b:8201",
  "protocol": "tcp-frame-json",
  "service_id": "service-b",
  "status": "available",
  "supported_versions": [
    "1"
  ],
  "version": "1"
},
{
  "capabilities": [
    "echo",
    "metadata",
    "sum"
  ],
  "endpoint": "service-c:8301",
  "protocol": "udp-crc-json",
  "service_id": "service-c",
  "status": "available",
  "supported_versions": [
    "1"
  ],
  "version": "1"
}
],
"status": "ok",
"uptime_ms": 1216
}
```

Service registry juga dirancang supaya aman ketika diakses oleh beberapa *request* secara bersamaan. Setiap operasi *read* dan perubahan data akan dilindungi menggunakan **asyncio.Lock**. Data yang diberikan kepada komponen lain juga berupa

copy dari data asli sehingga perubahan yang ada di luar *registry* tidak bisa secara langsung untuk mengubah *internal state*. Setiap perubahan status, versi, metadata, atau pendaftaran service baru akan disimpan secara otomatis ke persistent storage.

Proses penyimpanannya akan dilakukan secara atomik dengan melakukan *write* data ke *temporary file* terlebih dahulu. Kemudian data disini akan di-*flush* dan disinkronkan menggunakan **fsync**, lalu *temporary file* menggantikan *file registry* utama. Jika proses penyimpanan gagal, maka perubahan pada memori akan *rollback* ke kondisi sebelumnya. Sehingga dengan menggunakan cara ini, registry bisa mengurangi risiko file konfigurasi menjadi tidak lengkap atau rusak ketika terjadi kegagalan pada saat melakukan *write* data.

Data yang ada di dalam *registry* akan digunakan oleh *capability-based router* untuk mencari *service* yang mendukung operasi tertentu dan masih berada dalam kondisi *available*. Selain itu, *health monitor* juga akan memperbarui status setiap *service* pada registry berdasarkan hasil pemeriksaan yang dilakukan secara berkala. Informasi *registry* juga akan ditampilkan melalui *endpoint /services* dan */status*, sehingga kondisi dan konfigurasi *backend* bisa diamati dari luar *gateway*.

Strategi Translasi Protokol

Client mengirimkan *request* dalam format HTTP/JSON yang berisi **request_id**, **operation**, **arguments**, dan **options**. Setelah request tersebut divalidasi dan *service* tujuannya ditentukan, *gateway* akan meneruskan data yang sama kepada *adapter* melalui *interface execute()*. Sehingga *router* tidak perlu mengetahui detail *framing*, *header*, ataupun format *message* yang digunakan oleh setiap *backend*.

Setiap *adapter* akan bertanggung jawab untuk melakukan dua arah translasi. Pada arah *request*, *adapter* akan mengubah operasi dan argumen dari format *gateway* menjadi format protokol *backend*. Pada arah *response*, *adapter* melakukan deserialisasi, memvalidasi *response*, lalu mengubah hasilnya kembali menjadi objek hasil yang seragam. *Router* kemudian akan membungkus objek tersebut ke dalam *response gateway* yang berisi **request_id**, **status**, **service_id**, **operation**, **result**, dan **error**.

Berikut adalah contoh tabel dari pemetaan yang dilakukan.

<i>Data gateway</i>	Service A	Service B	Service C
operation	operation_name	opCode huruf kapital	Numeric operation code
arguments	operation_arguments	args	JSON payload
sum.values	-	args.numberList	values
Versi	Header X-Protocol-Version	Field version pada frame	Field version pada packet
Korelasi	X-Request-ID	64-bit request ID	64-bit request ID dan 32-bit seq number
Hasil	operation_result	resultData	result
Transport	HTTP/JSON	Raw TCP	UDP

- Translasi Service A

Untuk **Service A**, proses translasi relatif dilakukan secara langsung karena *gateway* dan *backend*-nya sama-sama menggunakan HTTP/JSON. Nama operasi dari *gateway* akan diubah menjadi **operation_name**, sedangkan argumen diubah menjadi **operation_arguments**. *Gateway* juga menambahkan

header **Content-Type**, **X-Protocol-Version**, dan **X-Request-ID**, kemudian akan mengirimkan request menggunakan metode POST ke *endpoint* **/v1/execute**.

Contohnya *request gateway* berikut.

```
{
  "request_id": "request-001",
  "operation": "uppercase",
  "arguments": {
    "value": "babel"
  },
  "options": {
    "preferred_service": "service-a",
    "timeout_ms": 2000
  }
}
```

akan diterjemahkan menjadi:

```
POST /v1/execute
Content-Type: application/json
X-Protocol-Version: 1
X-Request-ID: request-001

{
  "operation_name": "uppercase",
  "operation_arguments": {
    "value": "babel"
  }
}
```

- Translasi Service B

Service B menggunakan raw TCP dengan *binary framing*. Oleh karena itu, *adapter* akan terlebih dahulu mengubah nama operasi menjadi huruf kapital, seperti sum menjadi **SUM** dan reverse menjadi **REVERSE**. Khusus untuk operasi sum, *field values* dari *gateway* diubah menjadi **numberList** sesuai dengan kontrak **Service B**.

Payload JSON yang telah dibentuk kemudian akan dikonversi menjadi **byte UTF-8**. *Adapter* menambahkan *header* binary dengan ukuran 16 byte yang berisi *magic bytes* 0xBA 0xBE, versi, flags, panjang dari *payload*, dan request ID

64-bit. *Header* dan *payload* akan digabungkan menjadi satu *frame* sebelum dikirimkan melalui koneksi TCP.

Sebagai contoh, operasi:

```
{
  "operation": "sum",
  "arguments": {
    "values": [1, 2, 3]
  }
}
```

diubah menjadi payload:

```
{
  "opCode": "SUM",
  "args": {
    "numberList": [1, 2, 3]
  }
}
```

Ketika *response* diterima, *adapter* akan membaca tepat 16 byte untuk mendapatkan *header*, lalu membaca *payload* sesuai dengan panjang yang tercantum. *Adapter* memvalidasi *magic bytes*, versi, *flags*, ukuran dari *payload*, *request ID*, dan *serviceId*. Jika **Service B** mengembalikan **resultData.numericResult**, nilai tersebut akan dinormalisasi menjadi {"value": ...}. Sementara itu, **resultData.value** juga diubah ke bentuk yang sama.

- Translasi Service C

Service C menggunakan protokol UDP dengan *custom binary protocol*. *Adapter* akan terlebih dahulu men-*translate* nama operasi menjadi *operation code*, yaitu 1 untuk **echo**, 2 untuk **sum**, dan 3 untuk **metadata**. Argumen dari operasi akan diserialisasi menjadi JSON UTF-8 dan ditempatkan sebagai *payload packet*.

Sebelum nantinya dikirimkan, *payload* akan dilengkapi dengan *binary header* dengan panjang 20 byte. *Header* tersebut memuat *magic bytes* 0xC0 0xDE, versi protokol, *message type*, *sequence number*, request ID, *operation code*, *flags*, dan panjang *payload*. *Gateway* kemudian akan mulai menghitung checksum CRC32 seluruh bagian *header* dan *payload*, dan menambahkan nilai checksum tersebut ke empat byte terakhir *packet*.

Header 20 byte	JSON payload	CRC32 4 byte
----------------	--------------	--------------

Pada saat *response* diterima, *adapter* akan memvalidasi CRC32, *magic bytes*, versi, *message type*, *sequence number*, request ID, *operation code*, *flags*, dan panjang *payload*. *Packet* yang tidak valid tidak akan diteruskan kepada *client*. Setelah seluruh pemeriksaan telah berhasil dilakukan, payload JSON akan kemudian dideserialisasi dan *field result* digunakan sebagai hasil ternormalisasi.

Strategi Routing

Strategi Routing yang saya gunakan pada *gateway* ini adalah menggunakan **capability-based routing** yang artinya adalah pemilihan *backend* tidak hanya dilakukan berdasarkan nama *service* atau urutan yang tetap, tetapi berdasarkan kemampuan dari *service* untuk menjalankan operasi yang diminta. Informasi tentang *capability*, *health status*, *active version*, dan *metadata routing* didapatkan dari *service registry*.

Ketika sebuah *request* diterima, *router* akan mengambil semua *service* yang terdaftar lalu mem-filter-nya berdasarkan operasi pada *request*. Untuk *service* yang tidak memiliki *capability* tersebut tidak akan dimasukkan sebagai salah satu kandidat. Setelah itu, *service* yang berstatus **unavailable** juga akan dikeluarkan dari daftar kandidat supaya *request* tidak dikirimkan menuju *backend* yang telah diketahui sedang mengalami masalah.

Contohnya, jika *client* mengirim operasi *sum* dengan **preferred_service** berupa **service-a**, *router* tidak akan langsung menggunakan **Service A** karena *service* tersebut tidak memiliki *capability sum*. Router akan memilih *service* lain yang kompatibel, yaitu **Service B** atau **Service C**. Berdasarkan prioritas routing saat ini, **Service C** akan dipilih terlebih dahulu.

Setelah kandidatnya diperoleh, *router* akan mengurutkan *service* berdasarkan nilai prioritas yang mereka miliki. Berikut adalah konfigurasi *default* yang digunakan.

Operasi	Urutan service
echo	Service A, Service B, Service C
uppercase	Service A, Service B
sum	Service C, Service B
reverse	Service B
metadata	Service A, Service B, Service C

Client dapat menentukan *backend* yang diinginkan melalui *field* **options.preferred_service**. Akan tetapi, *preferred service* bersifat sebagai preferensi dan bukan *command* yang bersifat mutlak. *Service* tersebut hanya dipindahkan ke urutan pertama apabila memenuhi dua kondisi, yaitu memiliki capability yang diminta dan tidak berstatus **unavailable**.

Jika *preferred service* tidak mendukung operasi yang diminta, maka *router* akan mengabaikannya dan tetap memilih *service* yang kompatibel berdasarkan prioritas yang membuat mekanisme ini berguna untuk mencegah *request* diteruskan ke *backend* yang tidak mampu memproses operasi tersebut.

Contohnya:

```
{
  "request_id": "route-001",
  "operation": "sum",
  "arguments": {
    "values": [5, 15]
  },
  "options": {
    "preferred_service": "service-a",
    "timeout_ms": 2000
  }
}
```

Meskipun *client* memilih **service-a**, *request*-nya akan tetap diarahkan ke **service-c** karena **Service A** tidak mendukung operasi **sum**. Setelah *backend* berhasil dipilih, *router* akan menentukan versi adapter yang akan digunakan. Jika *client* mengisi **options.version_preference**, *router* akan mencoba menggunakan versi tersebut. Jika tidak, *router* menggunakan versi aktif yang tersimpan pada *service registry*.

Adapter kemudian akan diambil menggunakan kombinasi *service_id* dan *version*. Dengan demikian, *adapter* untuk beberapa versi bisa tersedia secara bersamaan. Jika *adapter* untuk versi yang diminta tidak tersedia, maka kandidat tersebut akan dilewati dan *router* dapat mencoba *service* lain yang kompatibel.

Selanjutnya, sebelum *request* dikirimkan, *router* memeriksa *circuit breaker* milik *service* tujuan. Ketika *circuit breaker* berada dalam keadaan OPEN, *request* tidak akan diteruskan ke *service* tersebut dan *router* mencoba kandidat berikutnya. Hal ini dilakukan untuk mencegah *gateway* secara terus-menerus mengirim *request* menuju *backend* yang sedang mengalami kegagalan berulang.

Dalam kondisi biasa, *circuit breaker* berada pada status **CLOSED**. Setelah mencapai lima kegagalan, statusnya akan berubah menjadi **OPEN**. Setelah *recovery time* selama lima detik, *circuit breaker* akan berubah menjadi **HALF_OPEN** dan mengizinkan satu *request* percobaan. Jika *request* tersebut berhasil, maka status akan kembali menjadi **CLOSED**. Jika masih gagal, maka *circuit breaker* kembali menjadi **OPEN**.

Seluruh proses *routing* akan menggunakan satu *global timeout budget* dari **options.timeout_ms**. *Router* akan menghitung *deadline* menggunakan *monotonic clock* ketika *request* mulai diproses. Sebelum mencoba setiap kandidat yang ada, *router* akan menghitung sisa waktu berdasarkan *deadline* tersebut.

Router juga dapat mencoba *backend* berikutnya hanya kalau kegagalan pada *backend* sebelumnya telah dinyatakan aman untuk melakukan *fallback*. Kegagalan tersebut bisa dianggap aman kalau *gateway* mengetahui bahwa *request* belum dikirimkan atau *backend* secara eksplisit menolak *request* sebelum menjalankan operasinya. Contohnya adalah kegagalan koneksi sebelum pengiriman, versi adapter yang tidak tersedia, *circuit breaker* yang terbuka, atau operasi yang tidak didukung.

Sebaliknya, apabila *timeout* terjadi setelah *request* mungkin telah dikirimkan, maka *router* akan menghentikan proses dan tidak mencoba *backend* lain. Hal ini dilakukan karena *gateway* tidak bisa untuk memastikan apakah *backend* pertama sudah menjalankan suatu operasi atau belum. Melakukan *fallback* dalam kondisi tersebut bisa menyebabkan operasi yang sama dieksekusi dua kali pada *backend* yang berbeda.

Model Concurrency

Model Concurrency yang digunakan pada *gateway* ini berbasis **asyncio**. Jadi setiap request HTTP akan diproses sebagai sebuah *coroutine* terpisah sehingga *gateway* bisa menangani banyak *request* secara bersamaan tanpa harus menunggu *request* sebelumnya selesai.

Untuk menjaga konsistensi dari data, komponen yang memiliki *shared state* seperti *service registry*, *circuit breaker*, *adapter manager*, serta request ID allocator akan dilindungi dengan menggunakan *lock*. Untuk UDP, jumlah *request* yang aktif akan dibatasi dengan *semaphore* supaya *traffic* yang berlebih tidak menyebabkan penggunaan *resource* yang tanpa batas. **Service B** menggunakan koneksi TCP yang terpisah untuk setiap *request*, sehingga *concurrency* tidak akan bergantung pada *multiplexing* satu koneksi.

```
services_for_echo = ("service-a", "service-b", "service-c")

tasks = [
    execute(
        client,
        f"demo-concurrent-{index:03d}",
        "echo",
        {"value": f"value-{index}"},
        preferred_service=services_for_echo[index % 3],
        timeout_ms=4000,
    )
    for index in range(60)
]

concurrent = await asyncio.gather(*tasks)

for index, (code, body) in enumerate(concurrent):
    assert code == 200
    assert body["status"] == "success"
    assert body["request_id"] ==
f"demo-concurrent-{index:03d}"
    assert body["result"] == {"value": f"value-{index}"}
```

asyncio.gather() akan menjalankan 60 request secara bersamaan. Setiap request memiliki `request_id` dan nilai berbeda, kemudian seluruh *response* akan diperiksa untuk memastikan tidak terjadi pertukaran hasil antar-*request*.

Pada gateway, setiap request juga diproses secara asynchronous:

```
@app.post("/execute", response_model=ExecuteResponse)
    async def execute_operation(request_data: ExecuteRequest):
        response = await router.execute_request(request_data)
        return JSONResponse(
            status_code=http_code,
            content=response.model_dump(),
        )
```

Pada *gateway*, *endpoint lexecute* dan proses *routing* menggunakan fungsi **async**, sehingga operasi dari jaringan tidak akan memblokir *request* lainnya. Khusus untuk komunikasi UDP, **asyncio.Semaphore(128)** digunakan untuk membatasi maksimal 128 request aktif supaya penggunaan resource tetap terkendali ketika traffic-nya sedang meningkat. *Semaphore* disini digunakan untuk mencegah jumlah *request* dari UDP yang aktif untuk tumbuh/*grow* tanpa batas.

```
class ReliableUDPTransport:
    def __init__(self, host: str, port: int, max_in_flight:
int = 128):
        self.host = host
        self.port = port
        self._in_flight = asyncio.Semaphore(max_in_flight)

    async def send_and_receive(self, ...):
        async with asyncio.timeout(timeout_budget_s):
            await self._in_flight.acquire()

        try:
            # Mengirim dan menerima UDP packet
            ...
        finally:
            self._in_flight.release()
```

Penanganan Kegagalan

Gateway ini akan menangani kegagalan dengan mengubah *error* dari berbagai *backend* menjadi sebuah format *error* yang seragam. *Error* tersebut memiliki **code**, **message**, dan **retryable**, sehingga *client* tidak perlu memahami bentuk *error* khusus dari **Service A**, **B**, maupun **C**.

Setiap *request* akan memiliki *global timeout budget*. Jika waktunya habis, maka *gateway* akan mengembalikan **BACKEND_TIMEOUT**. *Fallback* hanya dilakukan kalau *request* sudah dipastikan belum dieksekusi oleh *backend*. Jika status pengirimannya tidak bisa dipastikan, maka proses akan dihentikan untuk mencegah eksekusi berulang.

Gateway juga menggunakan *health monitor* dan *circuit breaker*. *Backend* yang gagal berulang kali akan ditandai dan status dari *circuit breaker* akan berubah menjadi **OPEN**, sehingga *request* berikutnya tidak langsung dikirimkan ke *backend* tersebut. Setelah waktu *recovery*, satu *request* percobaan diizinkan melalui status **HALF_OPEN**.

Response dari *backend* selalu divalidasi sebelum diteruskan kepada *client*. *Response* dengan request ID, versi, *framing*, *field* wajib, atau *checksum* yang salah akan ditolak sebagai **PROTOCOL_VALIDATION_ERROR**. Selain itu, *runtime adapter* akan dijalankan pada *subprocess* terpisah agar *adapter* yang crash atau berhenti merespons tidak men-*stop* proses utama *gateway*.

Keputusan dan Kompromi Teknis

Keputusan	Alasan	Kompromi Teknis
Menggunakan format <i>request</i> dan <i>response</i> internal yang seragam	Memisahkan logika gateway dari detail protokol setiap backend	Membutuhkan <i>adapter</i> khusus untuk setiap protokol dan versi
Menggunakan <i>capability-based routing</i>	<i>Request</i> hanya diarahkan ke <i>service</i> yang mendukung operasi terkait	Informasi <i>capability</i> dalam <i>registry</i> harus selalu dijaga agar tetap akurat
Menggunakan <i>asyncio</i> untuk <i>concurrency</i>	Cocok untuk banyak operasi jaringan yang bersifat <i>I/O-bound</i>	Operasi <i>blocking</i> harus dipisahkan agar tidak menghambat <i>event loop</i>
Membuat koneksi TCP baru untuk setiap <i>request</i> Service B	Implementasinya sederhana dan mencegah <i>response</i> tertukar antar- <i>request</i>	Memiliki <i>overhead</i> lebih besar dibanding <i>connection pooling</i> atau <i>multiplexing</i>
Menambahkan <i>reliability layer</i> di atas UDP	Memberikan <i>checksum</i> , korelasi, <i>timeout</i> , dan retransmisi	Belum mendukung <i>fragmentation</i> , <i>reassembly</i> , atau <i>sliding window multi-packet</i>
Menggunakan <i>global timeout budget</i>	Total waktu <i>request</i> tetap mengikuti batas yang diberikan <i>client</i>	<i>Backend</i> berikutnya hanya memperoleh sisa waktu setelah percobaan sebelumnya
Menggunakan <i>fallback</i> secara konservatif	Mencegah satu operasi untuk dieksekusi dua kali pada <i>backend</i> berbeda	<i>Availability</i> dapat berkurang ketika status eksekusi <i>backend</i> pertama tidak diketahui
Menyimpan <i>registry</i> dalam file JSON secara atomik	Sederhana, mudah diperiksa, dan tetap tersedia setelah restart	Belum cocok untuk beberapa <i>instance gateway</i> yang menulis <i>registry</i> secara bersamaan
Menggunakan <i>circuit breaker</i> per <i>service</i>	Mengisolasi <i>backend</i> yang mengalami kegagalan berulang	Status <i>circuit breaker</i> tersimpan di memori dan akan kembali ke awal setelah restart
Menggunakan operasi	Memastikan <i>transport</i> dan	Menambah <i>request</i>

<i>metadata</i> untuk <i>health check</i>	versi protokol benar-benar dapat digunakan	berkala menuju setiap <i>backend</i>
Mempertahankan beberapa versi <i>adapter</i>	Mendukung migrasi tanpa menghentikan <i>request</i> yang sedang berjalan	Membutuhkan pengelolaan <i>lifecycle</i> dan penggunaan resource yang lebih kompleks
Menjalankan <i>runtime adapter</i> dalam <i>subprocess</i>	<i>Crash</i> atau <i>blocking</i> pada plugin tidak membuat down <i>gateway</i> utama	Menambah <i>overhead</i> pembuatan proses dan komunikasi antarproses
Melakukan validasi response secara ketat	Mencegah <i>response</i> rusak atau salah korelasi diteruskan ke <i>client</i>	<i>Response backend</i> yang tidak mengikuti kontrak akan langsung ditolak
Menggunakan <i>container non-root</i> dan <i>read-only filesystem</i>	Mengurangi dampak apabila <i>gateway</i> atau <i>plugin</i> mengalami kompromi	Direktori yang membutuhkan penulisan harus disediakan melalui <i>volume</i> atau tmpfs

Keterbatasan Implementasi

Berikut merupakan beberapa keterbatasan implementasi dari *gateway* ini.

- *Backend* yang disediakan cuma mendukung protokol versi 1 sehingga migrasi langsung ke versi 2 belum bisa didemonstrasikan secara langsung menggunakan *backend* bawaan.
- *Reliable* UDP masih belum mendukung *fragmentation*, *reassembly*, dan *sliding window* untuk payload yang berukuran besar.
- **Service B** masih menggunakan koneksi TCP baru untuk setiap *request*-nya dan belum mendukung *connection pooling* atau *multiplexing*.
- *Service registry* menggunakan file JSON lokal sehingga masih belum mendukung sinkronisasi pada *deployment* dengan beberapa *instance gateway*.